

Technische Konzeption

Sharely — digitale Kleiderschrankverwaltung

Katharina Lutzky · Manuel Herrmann · Christian Maschke

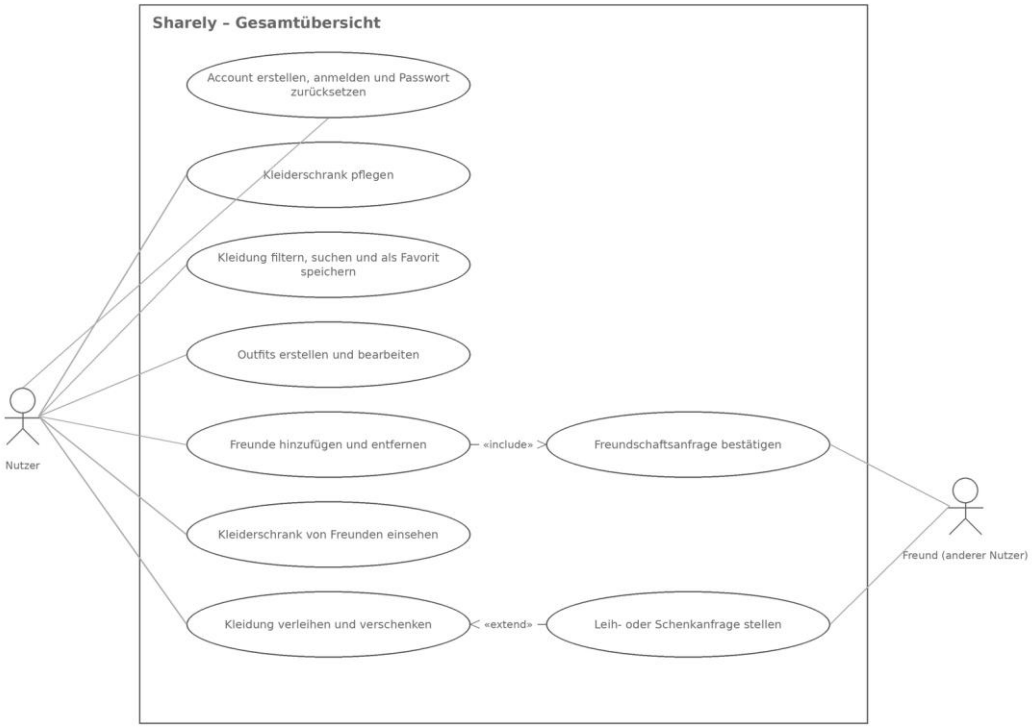
4. Oktober 2026

Inhalt

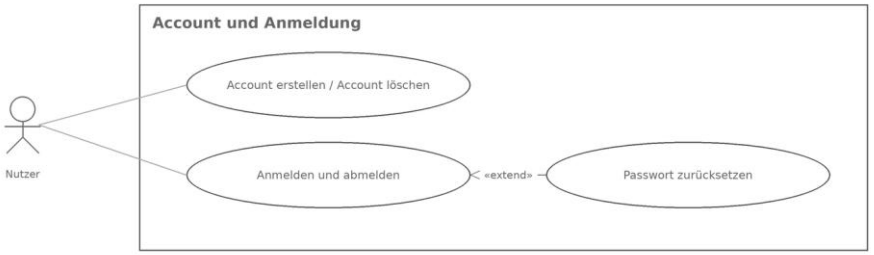
Inhalt 2
1. Use-Case-Diagramme 1
1.1 Gesamtübersicht 1
1.2 Account und Anmeldung 1
1.3 Kleiderschrank und Outfits 2
1.4 Freunde, Verleihen und Verschenken 2
2. Technologie-Stack 3
2.1 Welche Art von Anwendung soll entwickelt werden? 3
2.2 Welche Programmiersprachen und Frameworks werden verwendet? 3
2.3 Welche grundlegende Architektur wird gewählt? 4
3. Wireframes (UI/UX) 5
3.1 Anmeldung & Passwort 5
3.2 Schrank 6
3.3 Outfits 7
3.4 Verliehen 8
3.5 Freunde 9
3.6 Profil & Einstellungen 10

1. Use-Case-Diagramme

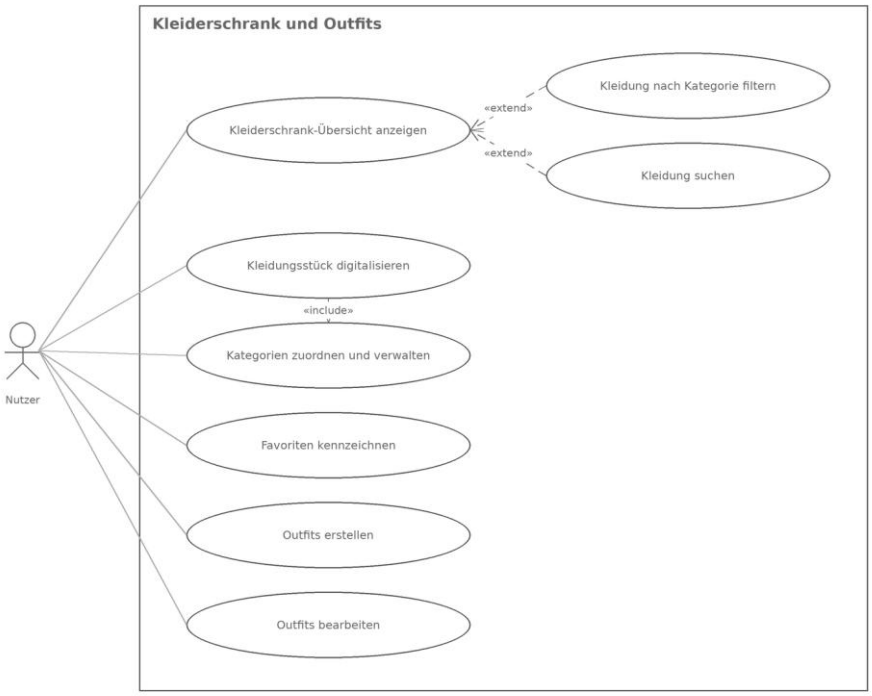
1.1 Gesamtübersicht



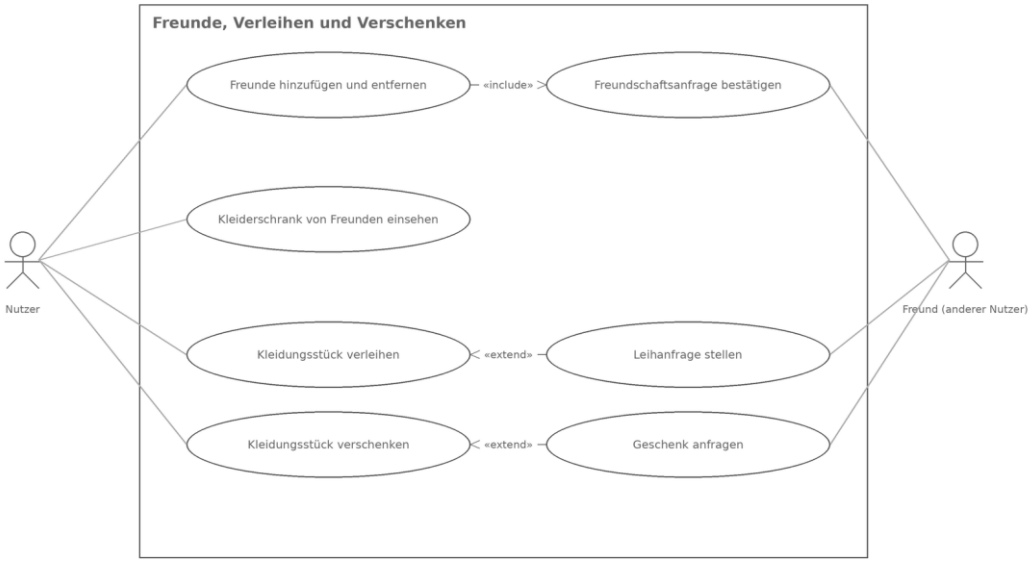
1.2 Account und Anmeldung



1.3 Kleiderschrank und Outfits



1.4 Freunde, Verleihen und Verschenken



2. Technologie-Stack

Hinweis: Der aktuelle Code im Repository (Flask + SQLite + serverseitige Templates) ist ein Experimentierstand aus der Testphase. Zielbild ist die unten beschriebene Aufteilung in FastAPI-Backend und JavaScript-Frontend.

2.1 Welche Art von Anwendung soll entwickelt werden?

Antwort: Web-App (im Browser laufende Single-Page-Anwendung, responsiv für Desktop und Smartphone).

Kriterium	Begründung
Eignung	Kleiderschrank, Outfits und Teilen mit Freunden sind klassische interaktive Funktionen mit Login — typisch für eine Web-App, nicht für eine reine Webseite.
Wartbarkeit	Eine Codebasis, auf die zugegriffen wird. Updates und Fixes sind für alle Nutzer sofort verfügbar.
Skalierbarkeit	Nutzerzahl wächst serverseitig mit; kein App-Store-Prozess nötig.
Entwicklungsaufwand	Im Team sind Web-Kenntnisse (HTML/CSS/JavaScript, Python) vorhanden. App-Entwicklung über z. B. den Play Store wäre zu aufwendig.
Aktualität	Moderne Browser unterstützen 3D-Grafik (WebGL) und Kamera-/Datei-Upload; optional lässt sich die Web-App später als PWA auf dem Handy „installieren“.

Keine einfache Website — das wäre für die interaktiven Funktionen zu statisch.

2.2 Welche Programmiersprachen und Frameworks werden verwendet?

Ebene	Technologie	Begründung
Backend	**Python mit FastAPI**	Das Team hat bereits viel Erfahrung damit. FastAPI ist modern und schnell, bietet umfangreiche Backend-Möglichkeiten und eignet sich sehr gut für die Kommunikation mit KI-APIs.
Frontend	**JavaScript und HTML/CSS**	Browser-Standard, auf allen Geräten nutzbar.
Frontend-Framework	**React oder Vue — Entscheidung noch offen**	Beide sind komponentenbasiert, weit verbreitet und gut dokumentiert. Komponenten (z. B. „Kleidungsstück-Karte“) werden einmal gebaut und überall wiederverwendet → bessere Wartbarkeit.
Datenbank	**MongoDB**	Objekte mit einer beliebigen Anzahl an Attributen lassen sich besser in eine dokumentorientierte Datenbank einpflegen. Datenstrukturen lassen sich flexibel ändern, und der Betrieb ist über den Cloud-Dienst MongoDB Atlas möglich. Persönliche Daten

		bleiben dabei in der eigenen Datenbank und nicht bei Drittanbietern.
Bildspeicher	**Serverseitige Speicherung**	Bilder gehören nicht in die Datenbank. Die einzelnen Items in MongoDB, denen ein Bild zugeordnet ist, verweisen auf das zugehörige Bild im Serverordner. Jeder Nutzer besitzt einen eigenen Bilderordner auf dem Server.

Kein SQLite wie im aktuellen experimentellen Stand, da es für mehrere Nutzer und flexible Kleidungsattribute weniger geeignet ist.

2.3 Welche grundlegende Architektur wird gewählt?

Antwort: Client-Server-Architektur mit einem modular aufgebauten Monolithen als Backend.

Kriterium	Begründung
Eignung	Client-Server trennt Darstellung (Browser, inkl. rechenintensiver 3D-Darstellung auf der Grafikkarte des Nutzers) von Logik und Daten (Server). Mehrere Nutzer greifen auf gemeinsame Daten zu (Teilen mit Freunden).
Wartbarkeit	Klare Schnittstelle (REST-API): Frontend und Backend können von verschiedenen Teammitgliedern parallel entwickelt werden. Innerhalb des Backends sorgen getrennte Module für Übersicht.
Skalierbarkeit	Für ein Projekt dieser Größe ausreichend; der Server kann bei Bedarf mehrfach gestartet werden. Durch die Modulaufteilung ließen sich einzelne Teile (z. B. KI-Import) später als eigener Dienst herauslösen.
Aktualität	REST-API + Single-Page-App ist heutiger Industriestandard.

3. Wireframes (UI/UX)

3.1 Anmeldung & Passwort

1. Anmeldung



2. Registrierung



3. Passwort zurücksetzen anfordern

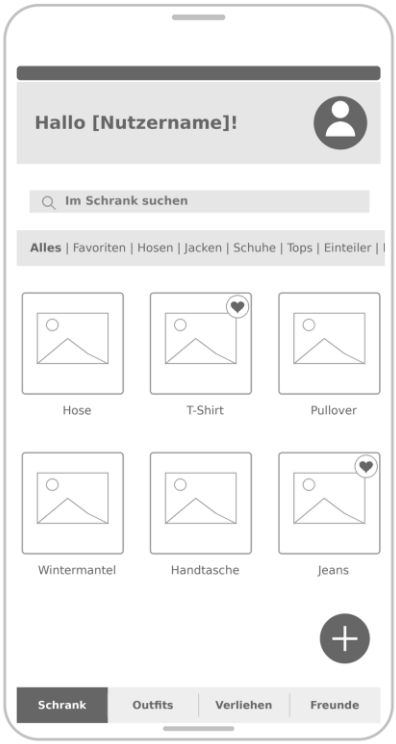


4. Neues Passwort setzen

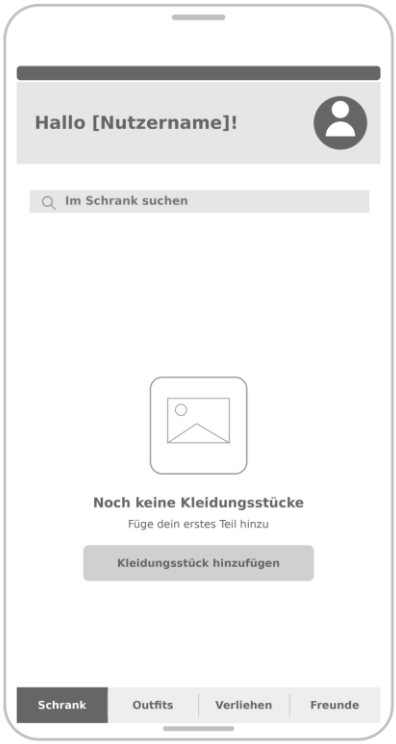


3.2 Schrank

5. Schrank (Hauptansicht)



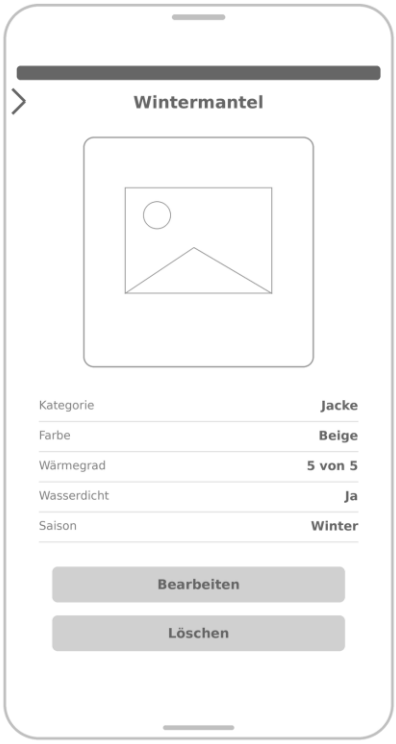
6. Schrank leer (Empty State)



7. Kleidungsstück hinzufügen



8. Kleidungsstück – Detailansicht



3.3 Outfits

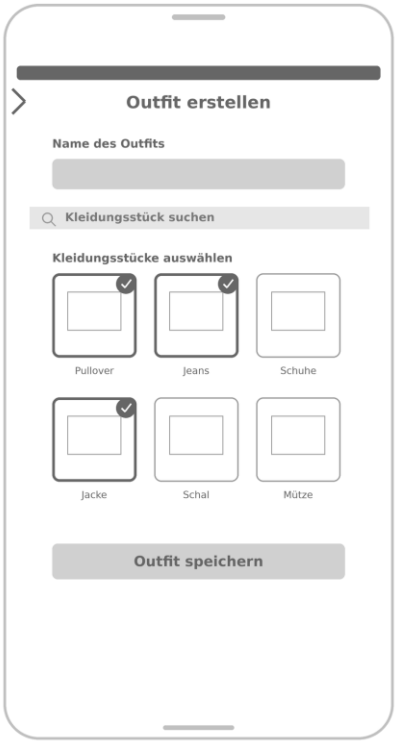
9. Outfits



10. Outfits leer (Empty State)



11. Outfit erstellen

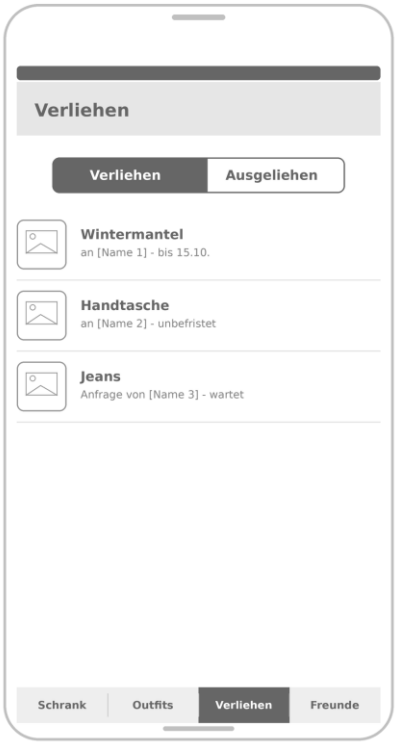


12. Outfit – Detailansicht

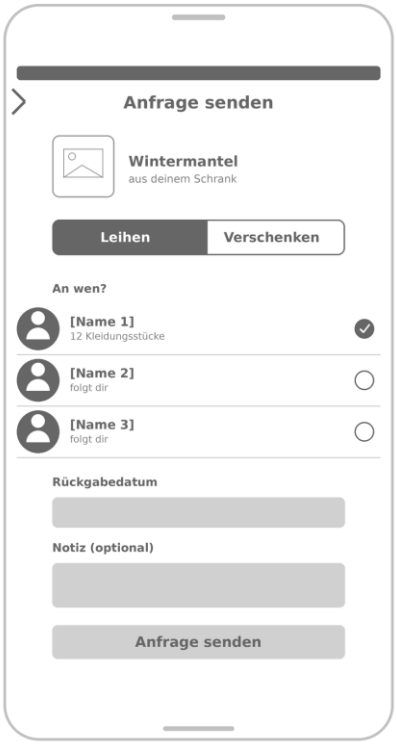


3.4 Verliehen

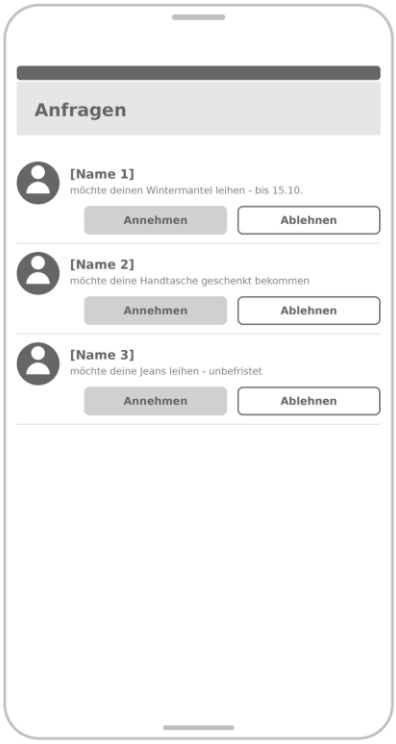
13. Verliehen



14. Anfrage stellen



15. Anfragen eingehend

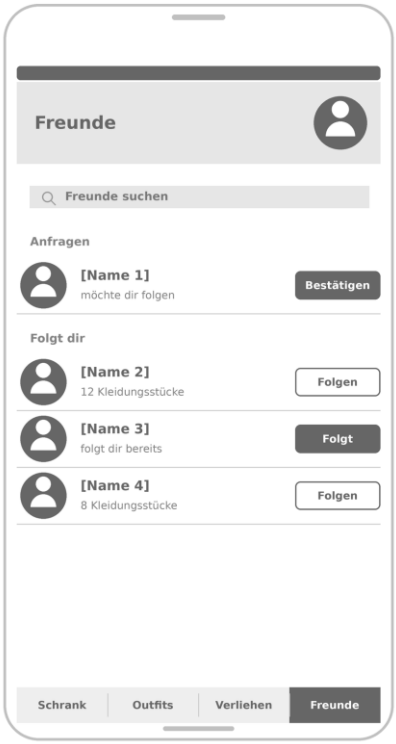


16. Leihe – Detailansicht



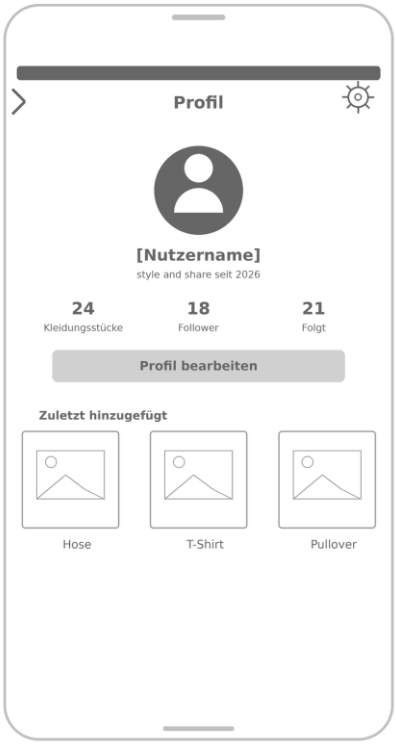
3.5 Freunde

17. Freunde

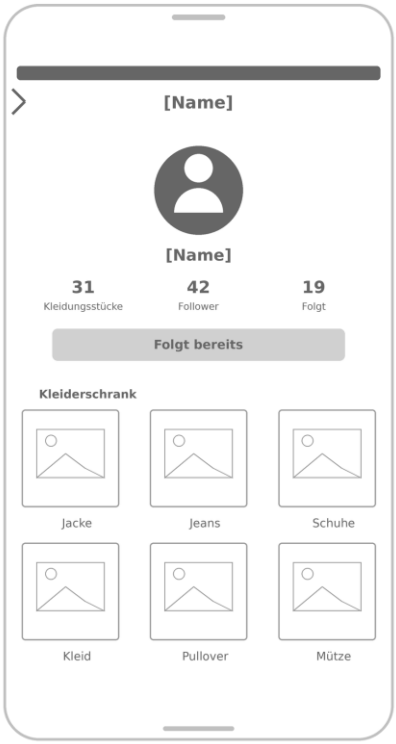


3.6 Profil & Einstellungen

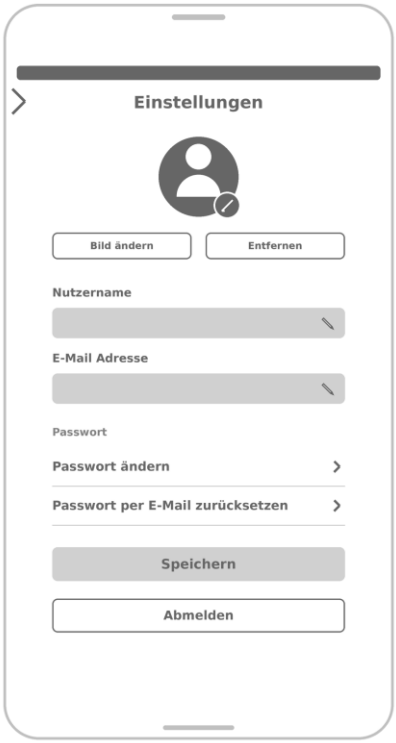
18. Eigenes Profil



19. Fremdes Profil



20. Einstellungen



21. Passwort ändern

